

FitTracker

Technical Architecture Specification, SRS Specification, and Mathematical Model of Automatic Progressive Overload

Lead Software Architect & Tech Lead

August 14, 2026

Abstract

This document serves as the formal software engineering specification for the mobile system **FitTracker**. Engineered under a *Local-First* paradigm and modular *Clean Architecture*, FitTracker automates resistance training management using an adaptive progressive overload engine based on mathematical 1RM (*1RepetitionMaximum*) estimation formulas and RPE/RIR autoregulation. It provides the Software Requirements Specification (SRS), UML use cases, SQLite database design, and the progressive algorithm mathematical model.

Contents

1	Introduction and Objectives	2
1.1	Context and Rationale	2
1.2	System Objectives	2
1.2.1	General Objective	2
1.2.2	Specific Objectives	2
1.3	Acknowledgments and Data Sources	2
2	Software Requirements Specification (SRS)	3
2.1	Functional Requirements (FR)	3
2.2	Non-Functional Requirements (NFR)	3
3	UML Use Cases	4
3.1	System Actors	4
3.2	Detailed Use Case Specification UC-01	4
4	Architecture and Database Design	5
4.1	System Architecture (Clean Architecture)	5
4.2	Entity-Relationship Model and SQLite Schema	5
5	Automatic Progressive Overload Algorithm	6
5.1	Mathematical Formulations for 1RM Estimation	6
5.2	Effective Volume and Automatic Progressive Overload	6
5.3	Next Session Load Progression Rule	6

1 Introduction and Objectives

1.1 Context and Rationale

In resistance training and muscle hypertrophy, the principle of **progressive overload** serves as the core foundation for driving long-term physiological adaptations. However, the vast majority of mobile applications on the market currently suffer from two critical limitations:

1. Strict reliance on Internet connectivity and cloud infrastructure (*Cloud-First*), which degrades the user experience in gyms with weak coverage or underground facilities.
2. Lack of deterministic automatic progressive overload algorithms based on physiological variables and quantitative metrics such as estimated 1RM and the RPE/RIR scale (*Rating of Perceived Exertion / Repetitions in Reserve*).

FitTracker is designed to solve both limitations through a **Local-First** mobile architecture powered by an embedded SQLite engine and an adaptive algorithmic progression model.

1.2 System Objectives

1.2.1 General Objective

Design, architect, and implement a cross-platform mobile application (React Native + Expo in TypeScript) characterized by 100% offline operation and Clean Architecture, capable of calculating and automatically increasing workload and volume for athletes based on historical performance.

1.2.2 Specific Objectives

- Implement an ultra-fast local persistence layer using SQLite, ensuring zero network latency and fault tolerance.
- Develop the mathematical 1RM (*1RepetitionMaximum*) estimation module by averaging classic Epley and Brzycki models with RIR correction.
- Provide a modular system architecture strictly separated into Domain, Use Cases, Repositories, and UI layers.
- Deliver formal academic technical documentation for engineering audit and research.

1.3 Acknowledgments and Data Sources

We formally acknowledge developer **Hasan Yıldırım** for creating and maintaining the open-source repository **exercises-dataset**. His structured JSON dataset provides the complete exercise database, metadata, and media elements powering the offline local seeding system in our persistence layer.

2 Software Requirements Specification (SRS)

This specification adapts the IEEE 830 standard for offline-first mobile systems.

2.1 Functional Requirements (FR)

Table 1: System Functional Requirements

Code	Name	Description
FR-01	Exercise Catalog Management	The system shall allow creating, modifying, and categorizing exercises by primary and secondary muscle groups.
FR-02	Routine Configuration	The system shall allow defining routines composed of exercise blocks, target rep ranges, and desired RPE.
FR-03	Live Set Logging	Allows logging weight (kg), reps (r), and RPE/RIR (R) in real-time with an automated rest timer.
FR-04	Automated Overload Calculation	The system shall compute weight/rep recommendations for the next session upon workout completion.
FR-05	1RM Estimation	The system shall compute current and historical estimated 1RM per session using validated mathematical models.
FR-06	Offline Nutrition Tracking	Daily logging of macronutrients (protein, carbs, fats) and total calories without requiring external APIs.
FR-07	Body Metrics Tracking	Tracking body weight, body fat percentage, and girth measurements.

2.2 Non-Functional Requirements (NFR)

Table 2: System Non-Functional Requirements

Code	Name	Metric / Acceptance Criteria
NFR-01	Offline Operability	100% of read and write operations must execute locally without an internet connection.
NFR-02	Write Performance	Inserting a completed set into SQLite must execute in under 50 ms .
NFR-03	Portability	The React Native (TypeScript) codebase must be portable across Android and iOS without logic modifications.
NFR-04	Data Integrity	SQLite database must use ACID transactions to prevent data corruption during unexpected app closures.

3 UML Use Cases

3.1 System Actors

- **Athlete / End User:** Individual executing workout routines, logging sets, and monitoring overload metrics.

3.2 Detailed Use Case Specification UC-01

Table 3: Use Case UC-01: Log Workout Set and Compute Progression

Use Case	UC-01: Log Workout Set and Compute Automatic Progression
Primary Actor	Athlete / End User
Preconditions	At least one exercise exists in the catalog and an active workout session is started.
Main Flow	1. The user selects the current exercise in the active session. 2. The system displays previously calculated target weight and reps. 3. The user inputs performed load (<i>kg</i>), completed reps, and RPE (6.0 – 10.0). 4. The user taps "Complete Set". 5. The system persists the set to SQLite, starts the rest timer, and increments set counter. 6. The overload engine updates the session's estimated 1RM.
Alternative Flow	<i>3a. User marks set as "Warmup Set":</i> - System saves the set but excludes it from effective volume and record 1RM calculations. <i>3b. User indicates muscle failure (RPE 10 / RIR 0):</i> - System adjusts fatigue accumulation factor for the next session calculation.
Postconditions	Set persisted in SQLite; rest timer triggers audio/vibration notification upon expiration.

4 Architecture and Database Design

4.1 System Architecture (Clean Architecture)

FitTracker implements Clean Architecture structured by functional feature modules (*Feature-First*).

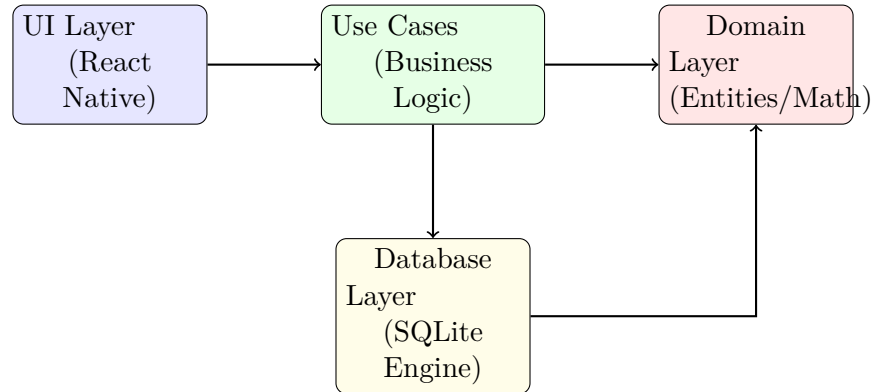


Figure 1: Layer Dependency Diagram in Clean Architecture.

4.2 Entity-Relationship Model and SQLite Schema

```
1 -- Exercises Table
2 CREATE TABLE exercises (
3     id TEXT PRIMARY KEY NOT NULL,
4     name TEXT NOT NULL,
5     category TEXT NOT NULL,
6     type TEXT NOT NULL,
7     equipment TEXT NOT NULL,
8     gif_url TEXT,
9     instructions TEXT
10 );
11
12 -- Workout Sessions Table
13 CREATE TABLE workout_sessions (
14     id TEXT PRIMARY KEY NOT NULL,
15     name TEXT NOT NULL,
16     date INTEGER NOT NULL,
17     notes TEXT
18 );
19
20 -- Logged Exercise Sets Table
21 CREATE TABLE exercise_sets (
22     id TEXT PRIMARY KEY NOT NULL,
23     session_id TEXT NOT NULL,
24     exercise_id TEXT NOT NULL,
25     set_order INTEGER NOT NULL,
26     weight_kg REAL NOT NULL,
27     reps INTEGER NOT NULL,
28     rpe REAL NOT NULL,
29     is_warmup INTEGER NOT NULL DEFAULT 0,
30     estimated_1rm REAL NOT NULL,
31     FOREIGN KEY(session_id) REFERENCES workout_sessions(id) ON DELETE CASCADE,
32     FOREIGN KEY(exercise_id) REFERENCES exercises(id) ON DELETE CASCADE
33 );
```

Listing 1: SQLite Relational DDL Schema

5 Automatic Progressive Overload Algorithm

5.1 Mathematical Formulations for 1RM Estimation

To quantify relative strength in a set of r repetitions executed with load w (expressed in kg) at a perceived exertion of $RPE \in [6.0, 10.0]$, we first calculate equivalent Repetitions in Reserve (RIR):

$$RIR = 10.0 - RPE \quad (1)$$

Estimated repetitions to absolute failure $r_{\max}$ are calculated as:

$$r_{\max} = r + RIR \quad (2)$$

We use a weighted combination of classic non-linear models by **Epley** and **Brzycki**:

1. Epley Formula:

$$1RM_{\text{Epley}} = w \cdot \left(1 + \frac{r_{\max}}{30}\right) \quad (3)$$

2. Brzycki Formula:

$$1RM_{\text{Brzycki}} = w \cdot \frac{36}{37 - r_{\max}} \quad (4)$$

The final estimated set 1RM ($1RM_{\text{est}}$) is computed by averaging both models:

$$1RM_{\text{est}} = \frac{1RM_{\text{Epley}} + 1RM_{\text{Brzycki}}}{2} \quad (5)$$

5.2 Effective Volume and Automatic Progressive Overload

The **Total Effective Volume** ($V_{\text{effective}}$) of a session for a given exercise, considering working sets only ($is_warmup = 0$), is expressed as:

$$V_{\text{effective}} = \sum_{i=1}^N w_i \cdot r_i \cdot \alpha(RPE_i) \quad (6)$$

where $\alpha(RPE_i)$ represents the fatigue weighting factor:

$$\alpha(RPE) = \begin{cases} 1.0 & \text{if } RPE \geq 8.0 \\ 0.85 & \text{if } 7.0 \leq RPE < 8.0 \\ 0.70 & \text{if } RPE < 7.0 \end{cases} \quad (7)$$

5.3 Next Session Load Progression Rule

The recommended load w_{next} for the subsequent session is adjusted based on session average RIR_{avg} and target repetition range $[R_{\min}, R_{\max}]$ completion:

$$w_{\text{next}} = \begin{cases} w_{\text{current}} + \Delta w & \text{if } r_i \geq R_{\max} \forall i \text{ and } RIR_{\text{avg}} \geq 1.5 \\ w_{\text{current}} & \text{if } R_{\min} \leq r_i < R_{\max} \text{ or } 0.5 \leq RIR_{\text{avg}} < 1.5 \\ w_{\text{current}} \cdot (1 - \beta) & \text{if } RIR_{\text{avg}} < 0.5 \text{ (deload or excessive fatigue)} \end{cases} \quad (8)$$

where Δw is the standard weight increment (2.5 kg for compound lifts, 1.25 kg for isolation exercises) and $\beta \in [0.05, 0.10]$ is the fatigue deload factor.